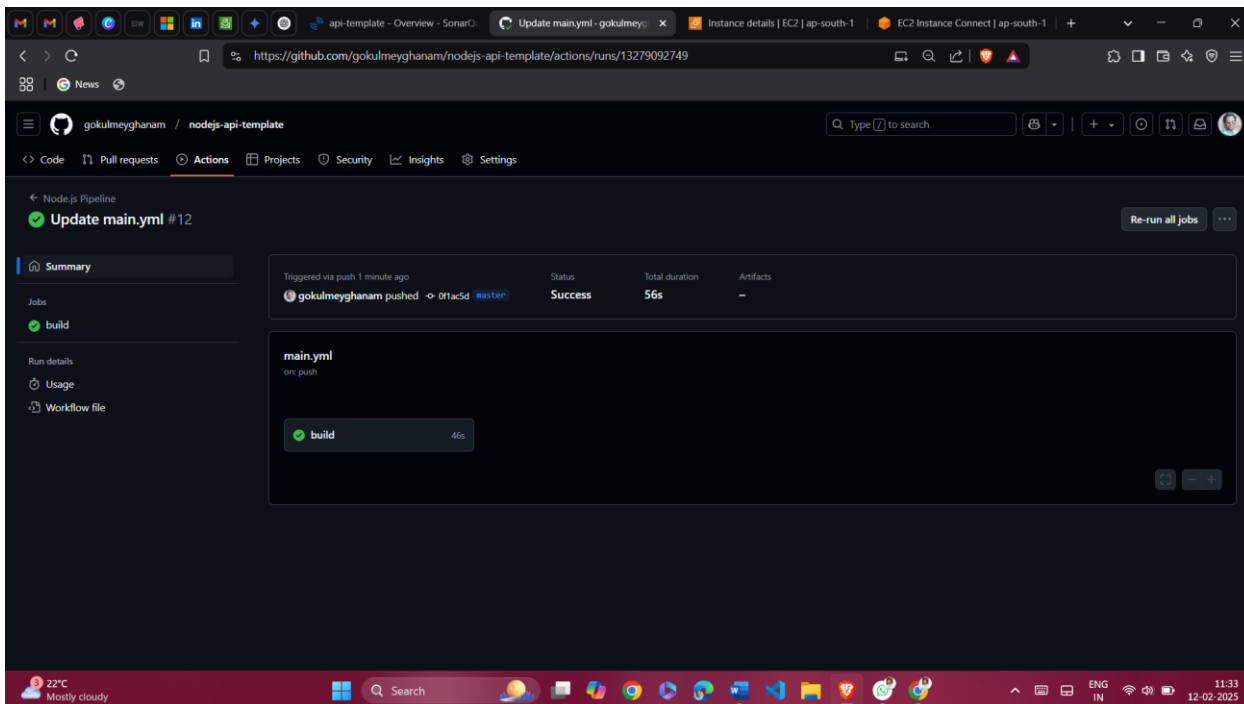
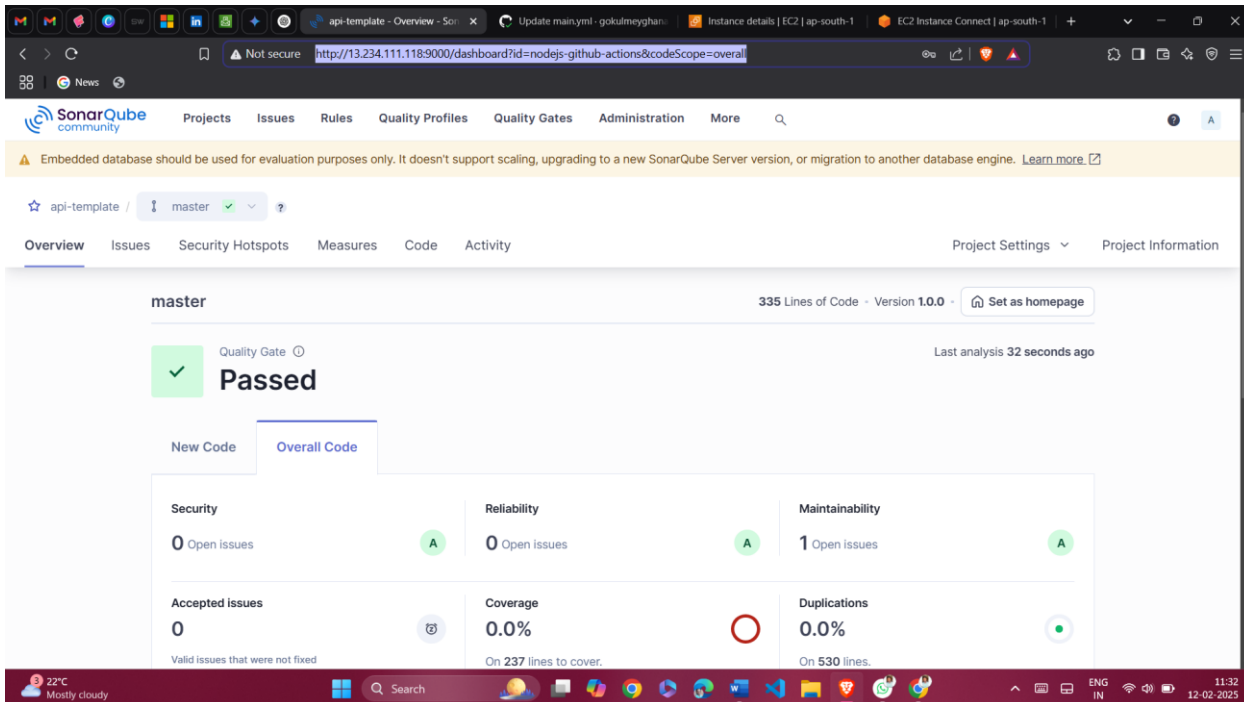


1) SONARQUBE INTEGRATION WITH GITHUB ACTION



SONAR_TOKEN: `sqa_ab599d18d05e274425025e4d07fe501fd189b46c`

GITHUB_REPO: - <https://github.com/gokulmeyghanam/nodejs-api-template>

docker run -d --name sonarqube -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true -p 9000:9000 sonarqube:latest

Documentation Refer: - <https://docs.sonarsource.com/sonarqube-server/10.8/try-out-sonarqube/>

name: Node.js Pipeline

on:

push:

branches:

- master

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v3

- name: Set up Node.js

uses: actions/setup-node@v3

with:

node-version: 20.x

cache: 'npm'

- name: Install dependencies

run: npm ci

- name: Build

run: npm run build --if-present

- name: Test

run: npm test

- name: Run SonarQube Analysis

env:

SONAR_HOST_URL: "http://13.234.111.118:9000/"

SONAR_TOKEN: \${ secrets.SONAR_TOKEN }

run: |

npm install -g sonarqube-scanner

sonar-scanner \

-Dsonar.projectKey=nodejs-github-actions \

-Dsonar.sources=. \

-Dsonar.host.url=\$SONAR_HOST_URL \

-Dsonar.login=\$SONAR_TOKEN

Generate SonarQube Authentication Token

1. Go to **My Account** (top-right corner in SonarQube UI).
2. Click **Security** → **Generate Tokens**.
3. Provide a token name (e.g., github-actions).
4. Click **Generate** and **copy** the token.

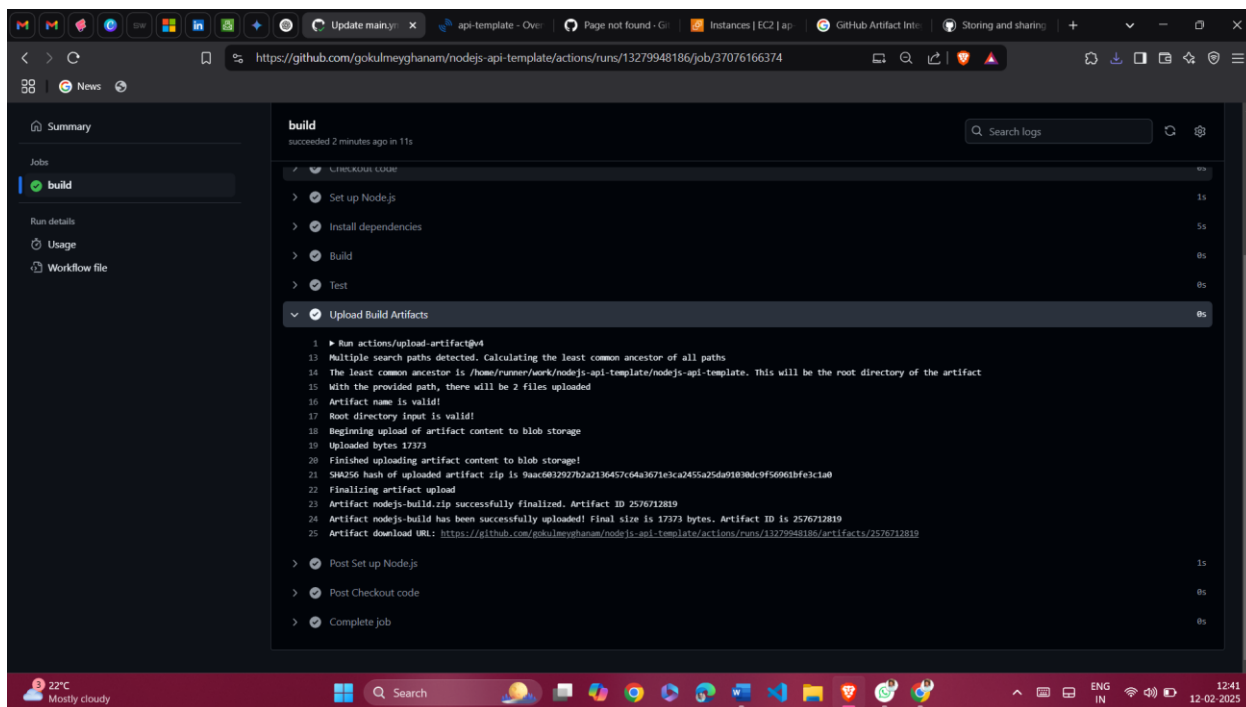
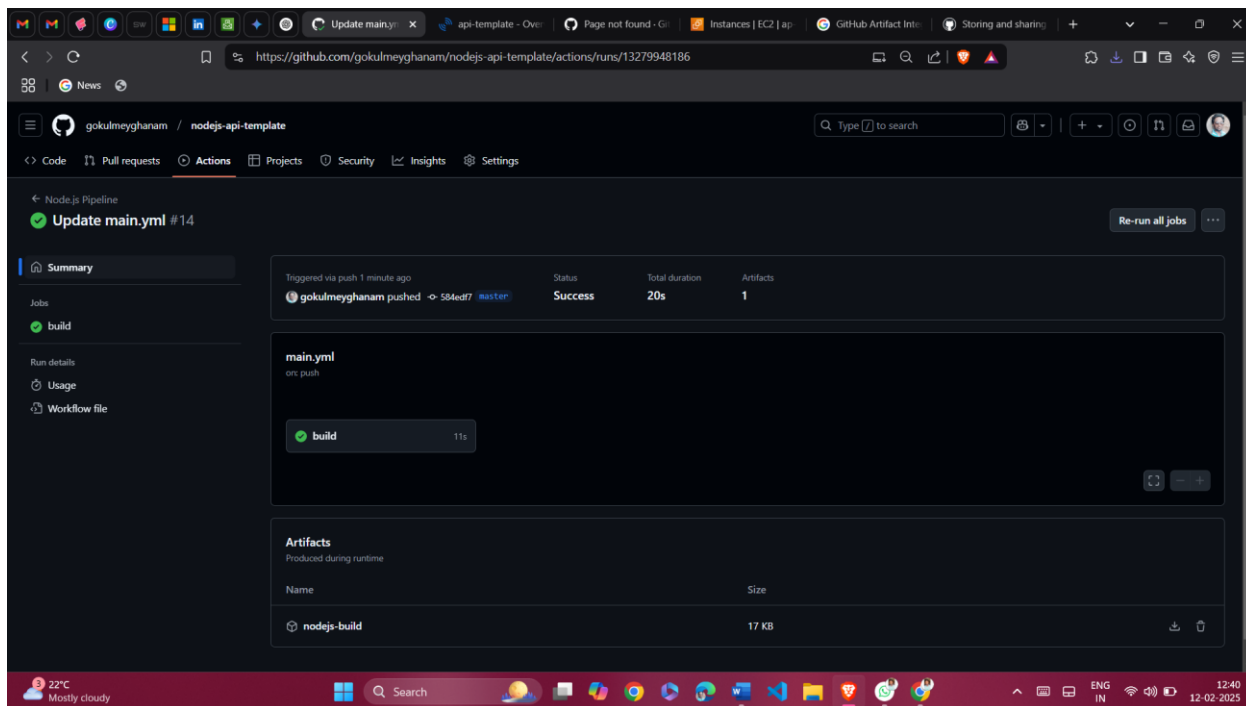
Create a SonarQube Project

1. Open **SonarQube UI** at https:ip of sonar
2. Navigate to **Projects** → **Create Project**.
3. Select **"Manually"** and enter:
 - **Project Key:** Example: nodejs-github-actions
 - **Display Name:** Example: Node.js App
4. Click **Create Project**.

Explanation of Workflow

- **Checks out the code** (actions/checkout@v3).
- **Sets up Node.js** (20.x version with npm cache).
- **Installs dependencies** (npm ci).
- **Builds the project** (npm run build --if-present).
- **Runs tests** (npm test).
- **Runs SonarQube Analysis:**
 - Installs sonarqube-scanner globally.
 - Runs sonar-scanner with:
 - **sonar.projectKey=nodejs-github-actions** should be created in sonarqube
 - **sonar.sources=.** Analyzes the entire project
 - **sonar.host.url=https://ip address** SonarQube server URL
 - **sonar.login=\$SONAR_TOKEN** Uses GitHub secret

2) GITHUB ARTIFACT INTEGRATION WITH GITHUB ACTION



Documentation Ref :- <https://docs.github.com/en/actions/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow>

Github Repo: - <https://github.com/gokulmeyghanam/nodejs-api-template>

name: Node.js Pipeline

on:

push:

branches:

- master

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Set up Node.js

uses: actions/setup-node@v4

with:

node-version: 20.x

cache: 'npm'

- name: Install dependencies

run: npm ci

- name: Build

run: npm run build --if-present

- name: Test

run: npm test

- name: Upload Build Artifacts

uses: actions/upload-artifact@v4

with:

name: nodejs-build

path: |

dist/

package.json

package-lock.json

retention-days: 5

3) DOCKER & DOCKERHUB INTEGRATION WITH GITHUB ACTION

The image shows two screenshots from a Windows desktop. The top screenshot is a GitHub Actions workflow run for the repository 'gokulmeyghanam/nodejs-api-template'. The workflow is named 'Update main.yml #18' and is part of a 'Node.js Pipeline'. It shows a successful run triggered by a push to the 'master' branch. The workflow consists of two jobs: 'build' (12s) and 'docker' (40s). The bottom screenshot is the Docker Hub 'Repositories' page for the user 'gokulmeyghanam'. It lists several public repositories, including 'nodejs-app', 'cloud-engine-lab', 'flask-app', 'shopping-cart', 'flask-mongodb-app', 'financeme', 'insuranceme', 'staragileprojectfinance', and 'springk8s'. The 'nodejs-app' repository is highlighted.

GitHub Actions Workflow Run Details:

- Repository: gokulmeyghanam / nodejs-api-template
- Workflow: Update main.yml #18
- Status: Success
- Total duration: 1m 7s
- Artifacts: 1
- Jobs: build (12s), docker (40s)

Docker Hub Repositories Table:

Name	Last Pushed	Contains	Visibility	Scout
gokulmeyghanam/nodejs-app	1 minute ago	IMAGE	Public	Inactive
gokulmeyghanam/cloud-engine-lab	13 days ago	IMAGE	Public	Inactive
gokulmeyghanam/flask-app	about 2 months ago	IMAGE	Public	Inactive
gokulmeyghanam/shopping-cart	5 months ago	IMAGE	Public	Inactive
gokulmeyghanam/flask-mongodb-app	6 months ago	IMAGE	Public	Inactive
gokulmeyghanam/financeme	6 months ago	IMAGE	Public	Inactive
gokulmeyghanam/insuranceme	8 months ago	IMAGE	Public	Inactive
gokulmeyghanam/staragileprojectfinance	9 months ago	IMAGE	Public	Inactive
gokulmeyghanam/springk8s	9 months ago	IMAGE	Public	Inactive

Github Repo: - <https://github.com/gokulmeyghanam/nodejs-api-template>

name: Node.js Pipeline

on:

push:

branches:

- master

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Set up Node.js
uses: actions/setup-node@v4
with:
node-version: 20.x
cache: 'npm'
- name: Install dependencies
run: npm ci
- name: Build
run: npm run build --if-present
- name: Test
run: npm test
- name: Upload Build Artifacts
uses: actions/upload-artifact@v4
with:
name: nodejs-build
path: |
dist/
package.json
package-lock.json
retention-days: 5

docker:

runs-on: ubuntu-latest

needs: build

steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Download Build Artifacts
uses: actions/download-artifact@v4
with:
name: nodejs-build
path: .
- name: Log in to Docker Hub
run: echo "\${{ secrets.DOCKER_PASSWORD }}" | docker login -u "\${{ secrets.DOCKER_USERNAME }}" --

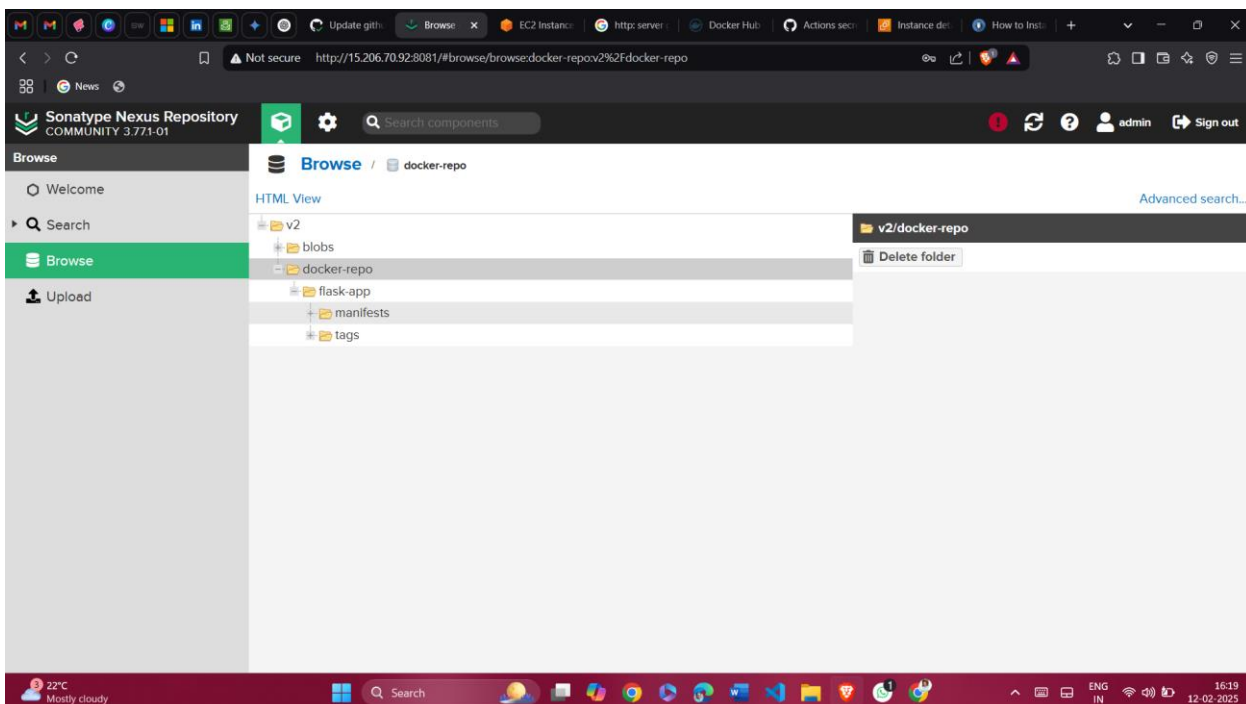
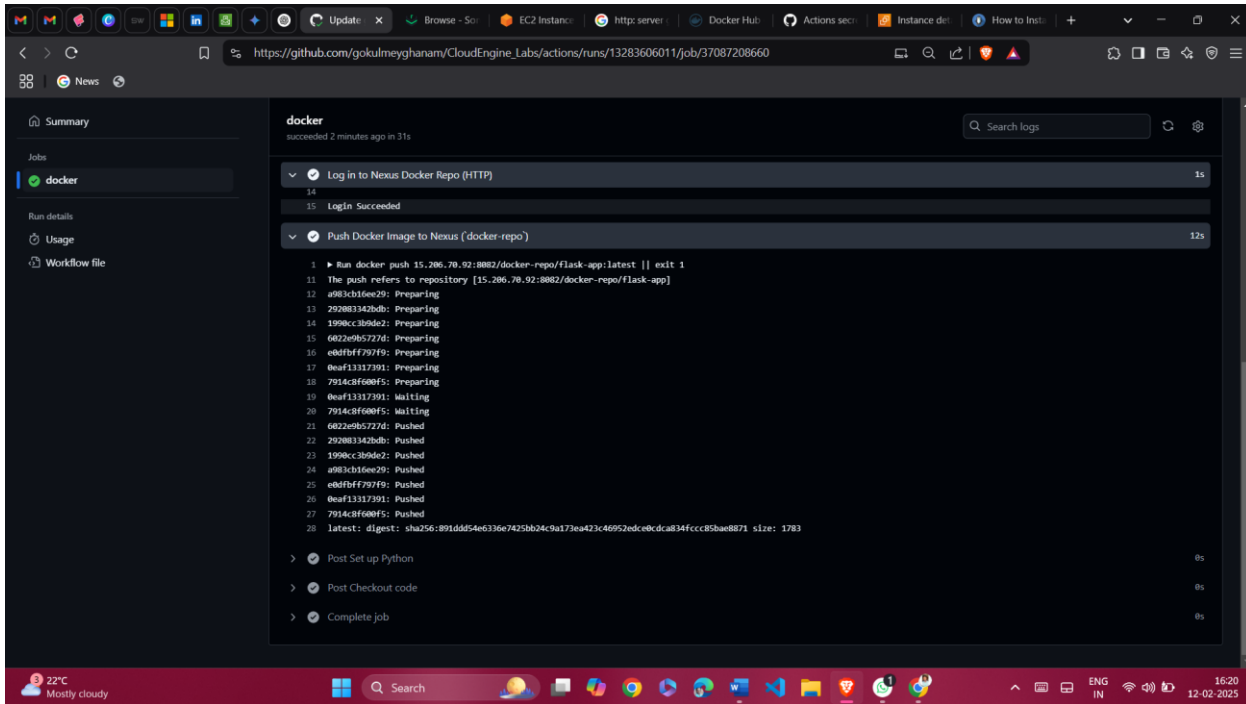
password-stdin

- name: Build Docker Image
run: docker build -t gokulmeyghanam/nodejs-app:latest .
- name: Push Docker Image to Docker Hub
run: docker push gokulmeyghanam/nodejs-app:latest

4) NEXUS INTEGRATION WITH GITHUB ACTION

Docker img: - `docker run -d -p 8081:8081 --name nexus sonatype/nexus3`

Documentation: - <https://hub.docker.com/r/sonatype/nexus3/#running>



name: Python Flask CI Pipeline

on:

push:

branches:

- main

jobs:

docker:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Set up Python

uses: actions/setup-python@v4

with:

python-version: 3.9

- name: Install Dependencies

run: pip install flask

- name: Build Docker Image

run: docker build -t gokulmeyghanam/flask-app:latest .

- name: Log in to Docker Hub

run: echo "Gokul@8668" | docker login -u "gokulmeyghanam" --password-stdin

- name: Push Docker Image to Docker Hub

run: docker push gokulmeyghanam/flask-app:latest

- name: Tag Docker Image for Nexus (`docker-repo`)

run: docker tag gokulmeyghanam/flask-app:latest 15.206.70.92:8082/docker-repo/flask-app:latest

- name: Allow HTTP for Nexus Docker Registry

run: |

echo '{ "insecure-registries": ["15.206.70.92:8082"] }' | sudo tee /etc/docker/daemon.json

sudo systemctl restart docker

sudo systemctl daemon-reexec

- name: Log in to Nexus Docker Repo (HTTP)

run: |

echo "Gokul@866819" | docker login -u "admin" --password-stdin http://15.206.70.92:8082 || exit 1

- name: Push Docker Image to Nexus (`docker-repo`)

run: docker push 15.206.70.92:8082/docker-repo/flask-app:latest || exit 1

NEED TO DONE IN NEXUS SERVER

If this Error Occurs -> http: server gave HTTP response to HTTPS client

Fix - 1

Docker **defaults to HTTPS** when pushing images. Since Nexus is running on **HTTP**, we must configure Docker to allow it:

Adds this configuration to /etc/docker/daemon.json:

```
{  
  "insecure-registries": ["15.206.70.92:8082"]  
}
```

sudo systemctl restart docker

Restarts the Docker service to apply changes.

Fix - 2

Fix: Ensure HTTP Login to Nexus

Instead of:

```
docker login -u admin -p Gokul@866819 15.206.70.92:8082
```

We now use **explicit HTTP**:

```
docker login -u admin -p Gokul@866819 http://15.206.70.92:8082
```

Fix - 3

```
docker stop nexus
```

```
docker rm nexus
```

```
docker run -d -p 8081:8081 -p 8082:8082 --name nexus sonatype/nexus3
```

Fix - 4

Log in to Nexus

Go to "Administration" → "Security" → "Realms".

Ensure "Docker Bearer Token Realm" is **enabled**.

Restart Nexus.

Fix - 5

Expose the Docker Repository Properly

If Nexus is running inside Docker, you need to expose port 8082.

Check your container with:

```
docker inspect nexus | grep HostPort
```

If port 8082 is missing, restart Nexus with correct port binding:

```
docker stop nexus
```

```
docker rm nexus
```

```
docker run -d -p 8081:8081 -p 8082:8082 --name nexus sonatype/nexus3
```

To Create Repo in Nexus

Log in to Nexus UI

Go to: Administration → Repositories.

Find your Docker repository (docker-repo) and check:

- It should be hosted (not proxy).
- The Port should be 8082.

- It should be enabled.
- Deployment Policy: Allow redeploy

Same for npm repo etc.

After Successfully Pushed artifact to Nexus. Follow Below Steps to verify

Method - 1

Verify Image Tags in Nexus

To check the available **image tags**, run this **API request**:

```
curl -X GET -u admin:Gokul@866819 "http://15.206.70.92:8082/v2/_catalog"
```

Expected Output:

```
{"repositories":["docker-repo/flask-app"]}
```

To list all tags for flask-app, run:

```
curl -X GET -u admin:Gokul@866819 http://15.206.70.92:8082/v2/docker-repo/flask-app/tags/list
```

Expected Output (Example):

```
{
  "name": "docker-repo/flask-app",
  "tags": ["latest", "v1.0"]
}
```

Method – 2

Use Nexus UI by navigate to repo we created earlier.

In the left sidebar, click on "Browse"

Find and select "docker-repo" (your Docker registry).

You should now see a list of pushed images.

Click on "flask-app" (or any pushed image) to view its details.

Here, you can see:

- The image name
- The available tags (e.g., latest)

Common cmd

For Manual Login - docker login -u admin -p Gokul@866819 <http://15.206.70.92:8082>

For Nexus Logs - docker logs nexus

For to check whether is running on port - sudo netstat -tulnp | grep 8082

Check Status for ip - curl -I <http://15.206.70.92:8082/v2/>

To Copy Nexus Admin Password – if NEXUS installer through docker image

```
docker exec -it <container_name_or_ID> /bin/bash
```

```
cat /nexus-data/admin.password
```

CI Pipeline for Python app (Checkout, build, docker build, docker push to Docker hub and nexus)

name: Python Flask CI/CD Pipeline

on:

push:

branches:

- master

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout Code

uses: actions/checkout@v3

- name: Set up Python

uses: actions/setup-python@v3

with:

python-version: 3.9

- name: Install Dependencies

run: pip install flask

- name: Run SonarQube Analysis

env:

SONAR_HOST_URL: "https://20.35.52.20:9000"

SONAR_TOKEN: "your-sonarqube-token" # Replace with actual token

run: |

pip install sonar-scanner-cli

sonar-scanner \

-Dsonar.projectKey=flask-app \

-Dsonar.sources=. \

-Dsonar.host.url=\$SONAR_HOST_URL \

-Dsonar.login=\$SONAR_TOKEN

- name: Build Docker Image

run: docker build -t gokulmeyghanam/flask-app:latest .

- name: Log in to Docker Hub

run: echo "Gokul@8668" | docker login -u "gokulmeyghanam" --password-stdin

- name: Push Docker Image to Docker Hub

```
run: docker push gokulmeyghanam/flask-app:latest
```

```
- name: Allow HTTP for Nexus Docker Registry
```

```
run: |
```

```
  echo '{ "insecure-registries": ["15.206.70.92:8082"] }' | sudo tee /etc/docker/daemon.json
```

```
  sudo systemctl restart docker
```

```
  sudo systemctl daemon-reexec
```

```
- name: Tag Docker Image for Nexus (`docker-repo`)
```

```
run: docker tag gokulmeyghanam/flask-app:latest 15.206.70.92:8082/docker-repo/flask-  
app:latest
```

```
- name: Log in to Nexus Docker Repo (HTTP)
```

```
run: |
```

```
  echo "Gokul@866819" | docker login -u "admin" --password-stdin http://15.206.70.92:8082 ||  
exit 1
```

```
- name: Push Docker Image to Nexus (`docker-repo`)
```

```
run: docker push 15.206.70.92:8082/docker-repo/flask-app:latest || exit 1
```